

Manual Scaling Test Results

1 Purpose

These tests were conducted to evaluate how manual Kubernetes scaling affects the performance of the ingestion service and the storage consumer. Two load-test dashboard pods were used so that the load generator would not become the limiting component before the platform.

Horizontal Pod Autoscaling (HPA) was not used. Only the replica count required for each comparison was changed.

2 Test Method

Two load generators were used at the same time for each test. Each generator was configured with the following parameters:

- 100,000 requests
- 5 concurrent workers
- 1,000 simulated devices
- 30-second timeout

A total of 200,000 requests was therefore sent for each configuration. The two generators were started approximately two seconds apart.

Combined throughput was calculated by dividing 200,000 requests by the time between the start of the first generator and the completion of the second generator. HTTP latency values were calculated as the average of the A and B dashboard results. Kafka, consumer, and PostgreSQL rates were also calculated as the average of the two monitoring summaries.

Each configuration was tested once under the same controlled test environment.

3 Test Configurations

Table 1: Manual scaling test configurations

Test	Ingestion Pods	Storage Consumers	Purpose
M07	1	1	High-load baseline
M08	2	1	Ingestion scaled to two pods
M09	4	1	Ingestion scaled to four pods
M10	2	2	Storage scaled to two consumers
M11	2	3	Storage scaled to three consumers
M12	2	4	Three-partition consumer limit

M07 was used as the baseline for ingestion scaling. M08 was used as the baseline for storage scaling because two ingestion pods were maintained in M10, M11, and M12.

4 Main Results

Table 2: Main performance results

Test	Combined req/s	HTTP p50 (ms)	HTTP p95 (ms)	HTTP p99 (ms)	Peak Lag	Kafka msg/s	Consumer rec/s	PostgreSQL rows/s
M07	1,775.30	5.09	7.99	10.40	1,592	1,805.57	1,805.03	1,804.76
M08	1,863.62	4.86	7.93	10.37	130	1,883.30	1,883.37	1,880.57
M09	1,797.45	5.09	8.08	11.53	124	1,800.58	1,800.56	1,790.11
M10	1,785.87	5.16	8.50	11.66	357	1,784.48	1,783.10	1,782.41
M11	1,690.01	5.44	9.08	12.54	179	1,697.78	1,696.65	1,690.47
M12	1,597.58	5.69	9.74	13.89	247	1,594.49	1,593.99	1,586.34

All 1,200,000 requests were completed successfully. An HTTP error rate of 0% was recorded in all 12 generator results.

All 100,000 records produced by each generator were stored in PostgreSQL. Therefore, 200,000 records were successfully stored for every configuration.

Final Kafka lag reached zero in every test, and no monitoring errors were recorded.

5 CPU and Batch Results

The CPU values shown below represent the total average CPU usage of all pods of the same type. For example, the ingestion CPU value for M09 represents the sum of the average CPU usage of all four ingestion pods.

Table 3: CPU usage and batch-size results

Test	Ingestion Avg. CPU (mCPU)	Storage Avg. CPU (mCPU)	Kafka Avg. CPU (mCPU)	Load Generators Avg. CPU (mCPU)	PostgreSQL Peak CPU (mCPU)	Average Batch Size
M07	598	347	554	1,353	496	11.72
M08	895	443	696	1,454	232	8.89
M09	1,143	436	771	1,529	203	9.57
M10	904	679	835	1,580	268	4.96
M11	920	891	857	1,571	363	3.35
M12	953	1,088	875	1,471	363	2.67

Kafka commit p95 was measured at 1 ms in every configuration. Batch-processing p95 and PostgreSQL-transaction p95 were measured at 10 ms in M08 and 25 ms in the other configurations.

6 Ingestion Scaling Analysis

One ingestion pod was used in M07, while two ingestion pods were used in M08. When M08 was compared with M07, the following changes were measured:

- Combined throughput was 4.97% higher.
- HTTP p50 latency was 4.68% lower.
- HTTP p95 latency was 0.77% lower.
- HTTP p99 latency was 0.29% lower.

The highest combined throughput in the test series was produced by M08 at 1,863.62 requests per second. A performance improvement was therefore achieved when the ingestion service was scaled from one pod to two pods under the distributed load.

Four ingestion pods were used in M09. Compared with M07, throughput was only 1.25% higher. Compared with M08, throughput was 3.55% lower. HTTP p99 latency was also higher than in both M07 and M08.

Total average ingestion CPU increased from 598 mCPU in M07 to 895 mCPU in M08 and 1,143 mCPU in M09. Throughput did not increase at the same rate. Therefore, more CPU was used by four ingestion pods without a useful performance improvement over two ingestion pods.

Traffic was not distributed perfectly equally between the ingestion pods. In M08, average CPU usage of approximately 308 mCPU and 579 mCPU was measured for the two pods. In M09, approximately 173, 377, 262, and 324 mCPU were measured for the four pods.

Persistent HTTP connections were used by the load generators. As a result, each connection remained attached to one ingestion pod, which affected how evenly traffic was distributed between pods.

7 Storage-Consumer Scaling Analysis

M08, with two ingestion pods and one storage consumer, was used as the baseline for storage scaling.

In M10, the number of storage consumers was increased from one to two. When M10 was compared with M08:

- Combined throughput was 4.17% lower.
- HTTP p95 latency was 7.18% higher.
- HTTP p99 latency was 12.41% higher.
- Total storage CPU usage was 53.18% higher.
- PostgreSQL peak CPU usage was 15.50% higher.

In M11, the number of storage consumers was increased to three. When M11 was compared with M08:

- Combined throughput was 9.32% lower.
- HTTP p95 latency was 14.45% higher.
- HTTP p99 latency was 20.95% higher.
- Total storage CPU usage was 100.85% higher.
- PostgreSQL peak CPU usage was 56.01% higher.

In M12, the number of storage consumers was increased to four. When M12 was compared with M08:

- Combined throughput was 14.28% lower.
- HTTP p95 latency was 22.76% higher.
- HTTP p99 latency was 33.95% higher.
- Total storage CPU usage was 145.26% higher.
- PostgreSQL peak CPU usage was 56.05% higher.

No performance improvement was produced by storage-consumer scaling for this workload. Higher total CPU usage was recorded as additional consumers were added, while the total processing rate was reduced.

The average batch size was also reduced from 8.89 records with one consumer to 4.96, 3.35, and 2.67 records as additional consumers were added.

With more consumers, fewer records were received by each consumer from its assigned partitions. Smaller batches were therefore processed more frequently, and more transaction and coordination work was required for each record.

The same PostgreSQL instance and Kafka broker were shared by all storage consumers. Additional concurrency and resource usage were created by increasing the consumer count, while the available database and broker capacity remained unchanged.

8 Kafka Partition Limit

The Kafka topic was configured with three partitions. Therefore, no more than three consumers in the storage-consumer group could be assigned active Kafka partitions at the same time.

Four storage-consumer pods were used in M12. Average CPU usage of approximately 351, 368, and 356 mCPU was measured for three of the consumers, while only approximately 16 mCPU was measured for the fourth consumer.

The fourth consumer was therefore mostly idle because no Kafka partition was available for it.

Compared with M11, the addition of the fourth consumer reduced combined throughput by another 5.47% and increased HTTP p95 latency by 7.26%. No additional Kafka consumption capacity could be provided by the fourth consumer.

9 Kafka Lag

The largest temporary Kafka lag was recorded in M07 at 1,592 messages. This represented less than 1% of the 200,000 messages sent during the configuration, and the complete backlog was processed after the generators finished.

Peak lag values between 124 and 357 messages were recorded in the other configurations. Every configuration finished with zero Kafka lag.

Consumer and PostgreSQL processing rates also remained close to the Kafka input rate. Therefore, the storage path was able to process the incoming load without a persistent backlog.

One storage consumer was already able to keep up with the workload. As a result, no backlog-related performance improvement was provided by additional consumers.

The two generators did not finish at exactly the same time. Final Kafka lag was therefore taken from the later result so that the complete paired load test was included in the final state.

10 Measurement Notes and Test Environment

The following conditions were part of the test setup and measurement method:

- The two load generators were started approximately two seconds apart.
- The reported latency percentiles were calculated as the average of the A and B dashboard percentiles. They were not recalculated from all 200,000 individual request timings.
- Kubernetes Metrics Server updates were less frequent than the dashboard sampling interval. CPU measurements should therefore be treated as sampled resource-usage values rather than continuous CPU traces.
- All services were executed on a single Docker Desktop Kubernetes node. The same host CPU, memory, and disk resources were shared by the complete system.

These conditions were applied consistently across the test configurations and were taken into account when the measurements were compared.

11 Conclusions

1. Two ingestion pods produced the highest combined throughput at 1,863.62 requests per second.
2. Compared with one ingestion pod, two ingestion pods increased combined throughput by 4.97% and reduced HTTP p50, p95, and p99 latency.

3. Four ingestion pods did not improve performance over two ingestion pods and required more total ingestion CPU.
4. One storage consumer was sufficient for this workload. Kafka lag returned to zero in every configuration, and the consumer processing rate remained close to the Kafka input rate.
5. Additional storage consumers reduced average batch size, increased total CPU usage, increased HTTP latency, and reduced combined throughput.
6. A fourth storage consumer did not provide additional Kafka consumption capacity because the topic contained only three partitions.
7. The platform remained reliable throughout the tests. All 1.2 million requests were accepted and stored successfully, no HTTP errors were recorded, and final Kafka lag was zero in every configuration.

For this workload, the best-performing configuration was two ingestion pods with one storage consumer. Scaling beyond this configuration added resource usage and coordination overhead without providing additional useful throughput.